Abstraction, Polymorphism, Interfaces, and Object Relationships

Made with GAMMA

Practical Example: Inheritance vs Composition

Real Software - Notification System

Inheritance creates rigid "is-a" relationships. Composition creates flexible "has-a" systems.

Bad: Inheritance (Rigid)

```
public class EmailNotifier
{
    public void Send() { }
}

public class SmsNotifier : EmailNotifier { }
// Forced inheritance
```

Good: Composition (Flexible)

```
public interface INotificationChannel
{
    void Send(string message, string recipient);
}

public class EmailChannel : INotificationChannel { }
public class SmsChannel : INotificationChannel { }
public class SlackChannel : INotificationChannel { }

public class NotificationService
{
    private readonly List _channels;

    public NotificationService(IEnumerable channels)
    {
        _channels = channels.ToList();
    }

    public void NotifyAll(string message, string recipient)
    {
        foreach (var channel in _channels)
        {
            channel.Send(message, recipient);
        }
    }
}

// Usage
var email = new EmailChannel();
var sms = new SmsChannel();
var slack = new SlackChannel();

var channels = new List<INotificationChannel> { email, sms, slack };
var service = new NotificationService(channels);
service.NotifyAll("System Update", "User_01");
```

Key Takeaway: Favor composition over inheritance for flexibility and testability.

Made with GAMMA

Compile-time Polymorphism

Method Overloading

Same method name, different signatures. Resolved at compile time.

```
public class Calculator
{
    // Overload 1: Two integers
    public void Add(int a, int b) { Console.WriteLine(a + b); }

    // Overload 2: Three integers
    public void Add(int a, int b, int c) { Console.WriteLine(a + b + c); }

    // Overload 3: Doubles
    public void Add(double a, double b) { Console.WriteLine(a + b); }

    // Overload 4: Array parameter
    public void Add(params int[] numbers) { Console.WriteLine(numbers.Sum()); }

    // Overload 5: Different parameter types
    public void Add(string a, string b) { Console.WriteLine(a + b); }
}

// Usage
var calc = new Calculator();
calc.Add(2, 3);              // int version
calc.Add(2.5, 3.5);          // double version
calc.Add("Hello", "World");  // string version
calc.Add(1, 2, 3, 4, 5);     // params version
```

Made with GAMMA

Runtime Polymorphism

Virtual/Override

Behavior determined at runtime based on actual object type.

```
public class Employee
{
  public string Name { get; set; } = string.Empty;
  public decimal BaseSalary { get; set; }

  public virtual decimal CalculateSalary() { return BaseSalary; }
  public virtual void Work() { Console.WriteLine($"{Name} is working."); }
}

public class Developer : Employee
{
  public decimal Bonus { get; set; }

  public override decimal CalculateSalary() { return BaseSalary + Bonus; }
  public override void Work() { Console.WriteLine($"{Name} is writing code."); }
}

public class Manager : Employee
{
  public decimal TeamBonus { get; set; }

  public override decimal CalculateSalary() { return BaseSalary + TeamBonus; }
  public override void Work() { Console.WriteLine($"{Name} is managing the team."); }
}

// Usage - Runtime Polymorphism
Employee emp1 = new Developer { Name = "Alice", BaseSalary = 5000, Bonus = 1000 };
Employee emp2 = new Manager { Name = "Bob", BaseSalary = 6000, TeamBonus = 2000 };

List<Employee> employees = new() { emp1, emp2 };

foreach (var emp in employees)
{
  Console.WriteLine($"{emp.Name}: ${emp.CalculateSalary()}");
  emp.Work();
}

// Output:
// Alice: $6000
// Alice is writing code.
// Bob: $8000
// Bob is managing the team.
```

Made with GAMMA

Abstract Classes and Methods

Define template/blueprint. Cannot instantiate. Forces implementation in derived classes.

```
public abstract class PaymentProcessor
{
    public void ValidateAmount(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Amount must be positive");
    }

    public abstract void ProcessPayment(decimal amount, string currency);
    public abstract string ProcessorName { get; }

    public virtual void LogTransaction(string transactionId)
    {
        Console.WriteLine($"[{ProcessorName}] Transaction logged: {transactionId}");
    }
}

public class StripeProcessor : PaymentProcessor
{
    public override string ProcessorName => "Stripe";

    public override void ProcessPayment(decimal amount, string currency)
    {
        ValidateAmount(amount);
        Console.WriteLine($"Processing ${amount} {currency} via Stripe API");
    }

    public override void LogTransaction(string transactionId)
    {
        Console.WriteLine($"Stripe Audit: {transactionId}");
    }
}

public class PayPalProcessor : PaymentProcessor
{
    public override string ProcessorName => "PayPal";

    public override void ProcessPayment(decimal amount, string currency)
    {
        ValidateAmount(amount);
        Console.WriteLine($"Processing ${amount} {currency} via PayPal SDK");
    }
}

// Usage
PaymentProcessor stripe = new StripeProcessor();
stripe.ProcessPayment(100, "USD");

// PaymentProcessor pp = new PaymentProcessor(); // ERROR: Cannot instantiate
```

Made with GAMMA

Interfaces

Definition, Implementation, Explicit Usage

Contracts that define what a class CAN DO.

```
public interface IAuthenticable
{
    bool Login(string username, string password);
    void Logout();
}

public class User : IAuthenticable
{
    public string? Username { get; private set; }

    public bool Login(string username, string password)
    {
        if (password == "12345")
        {
            Username = username;
            Console.WriteLine($"{Username} logged in successfully.");
            return true;
        }
        return false;
    }

    public void Logout()
    {
        Console.WriteLine($"{Username} has logged out.");
    }
}

// Usage
IAuthenticable auth = new User();
auth.Login("Admin", "12345");
auth.Logout();
```

Made with GAMMA

Interface Segregation Principle

Default Methods

Keep interfaces small and focused. C# 8.0+ supports default implementations.

```
public interface IWorkable { void Work(); }
public interface IEatable  { void Eat(); }

public interface ILogger
{
    void Log(string message);
    void LogError(string error) => Log($"[ERROR]: {error}");
}

public class ConsoleLogger : ILogger
{
    public void Log(string message) => Console.WriteLine(message);
}

public class Robot : IWorkable
{
    public void Work() => Console.WriteLine("Robot is processing...");
}

public class Human : IWorkable, IEatable
{
    public void Work() => Console.WriteLine("Human is coding...");
    public void Eat()  => Console.WriteLine("Human is eating lunch.");
}
```

Made with GAMMA

Interfaces vs Abstract Classes

Feature	Abstract Class	Interface
Instantiation	No	No
Inheritance	Single only	Multiple allowed
Implementation	Can have concrete methods	Default methods (C# 8+)
Fields	Yes	No (static allowed in C# 8+)
Access Modifiers	Any	Public by default

Made with GAMMA

Association, Aggregation, and Composition

Object relationships from weak to strong

Association (Uses-a)

Weakest relationship

```
public class Student
{
    public string? Name { get; set; }
}

public class Course
{
    public void Enroll(Student student)
    {
        Console.WriteLine($"{student.Name} enrolled");
    }
}

var student = new Student { Name = "Alice" };
var course = new Course();
course.Enroll(student);
```

Aggregation (Has-a)

Whole/part, independent lifetimes

```
public class Employee
{
    public string Name { get; set; }
}

public class Department
{
    public string Name { get; set; }
    public List<Employee> Employees { get; set; } = new();
}

var emp = new Employee { Name = "Bob" };
var dept = new Department { Name = "IT" };
dept.Employees.Add(emp);
// If dept is deleted, emp still exists
```

Composition (Has-a)

Whole/part, dependent lifetime

```
public class House
{
    public List Rooms { get; } = new();

    public House()
    {
        Rooms.Add(new Room("Living Room"));
        Rooms.Add(new Room("Kitchen"));
    }

    public class Room
    {
        public string Name { get; }
        public Room(string name) => Name = name;
    }
}

var house = new House();
// If house is destroyed, rooms are destroyed too
```

Made with GAMMA